Secant Method Root Solver in MATLAB

Numerical Methods Portfolio Project

Author

Ehsan Shakil Ahmad

Developed Using

MATLAB Online

Project Date

June 2026

Project Overview

This project presents the development of a MATLAB program that implements the Secant Method for solving nonlinear equations. The program calculates the root of a given function through iterative approximation using two initial guesses, displays the convergence process, and visualizes the result graphically. The project demonstrates the application of derivative-free numerical methods and MATLAB programming in engineering computation.

Keywords: Numerical Methods, MATLAB, Root Finding, Secant Method, *Derivative-Free Root Finding*, Engineering Computation

TABLE OF CONTENTS

1 OBJECTIVES

- To understand the theory behind the Secant Method.
- To develop a MATLAB program for solving nonlinear equations using the Secant Method.
- To implement convergence monitoring and numerical stability checks.
- To visualize the computed root using graphical representation.
- To investigate the effect of different initial guesses on convergence behavior.
- To compare the Secant Method with the Bisection and Newton-Raphson Methods.
- To strengthen MATLAB programming and numerical analysis skills through practical implementation.

2 THEORY

The Secant Method is an iterative numerical technique for solving nonlinear equations. Unlike the Newton-Raphson Method, it does not require evaluation of the derivative of the function.

For the nonlinear equation

$$f(x) = x^3 - x - 2$$

the Secant Method generates successive approximations using:

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$$

The method approximates the derivative using two previous points on the function curve. Iterations continue until

$$| x_{n+1} - x_n | < tol$$

The Secant Method generally converges faster than the Bisection Method and often approaches the efficiency of the Newton-Raphson Method while avoiding explicit derivative calculations.

3 NOMENCLATURE TABLE

Symbol	Description
x_0	First initial guess
x_1	Second initial guess
x_n	Current approximation
x_{n+1}	Next approximation
f(x)	Function
tol	Convergence tolerance
iter	Iteration counter

4 CODE IMPROVEMENT

Several improvements were incorporated into the MATLAB implementation to improve flexibility, reliability, and usability.

Instead of fixed starting values, the program accepts two user-defined initial guesses. This allows different convergence scenarios to be investigated without modifying the source code.

A denominator-checking condition was added before each Secant iteration. Since the Secant formula contains the term

$$f(x_n) - f(x_{n-1})$$

very small denominator values may lead to numerical instability. Therefore, the program terminates execution and displays an error message when the denominator becomes excessively small.

The program displays each approximation in a convergence table, allowing the convergence process to be monitored. Graphical visualization was also incorporated to verify the computed root and illustrate convergence behavior.

4.1 COMPUTATIONAL EFFICIENCY

The Secant Method is computationally efficient because it approximates derivative information using function values from previous iterations. Unlike Newton-Raphson, no analytical derivative is required.

For the nonlinear equation

$$f(x) = x^3 - x - 2$$

the Secant Method converged to the root

$$x = 1.521380$$

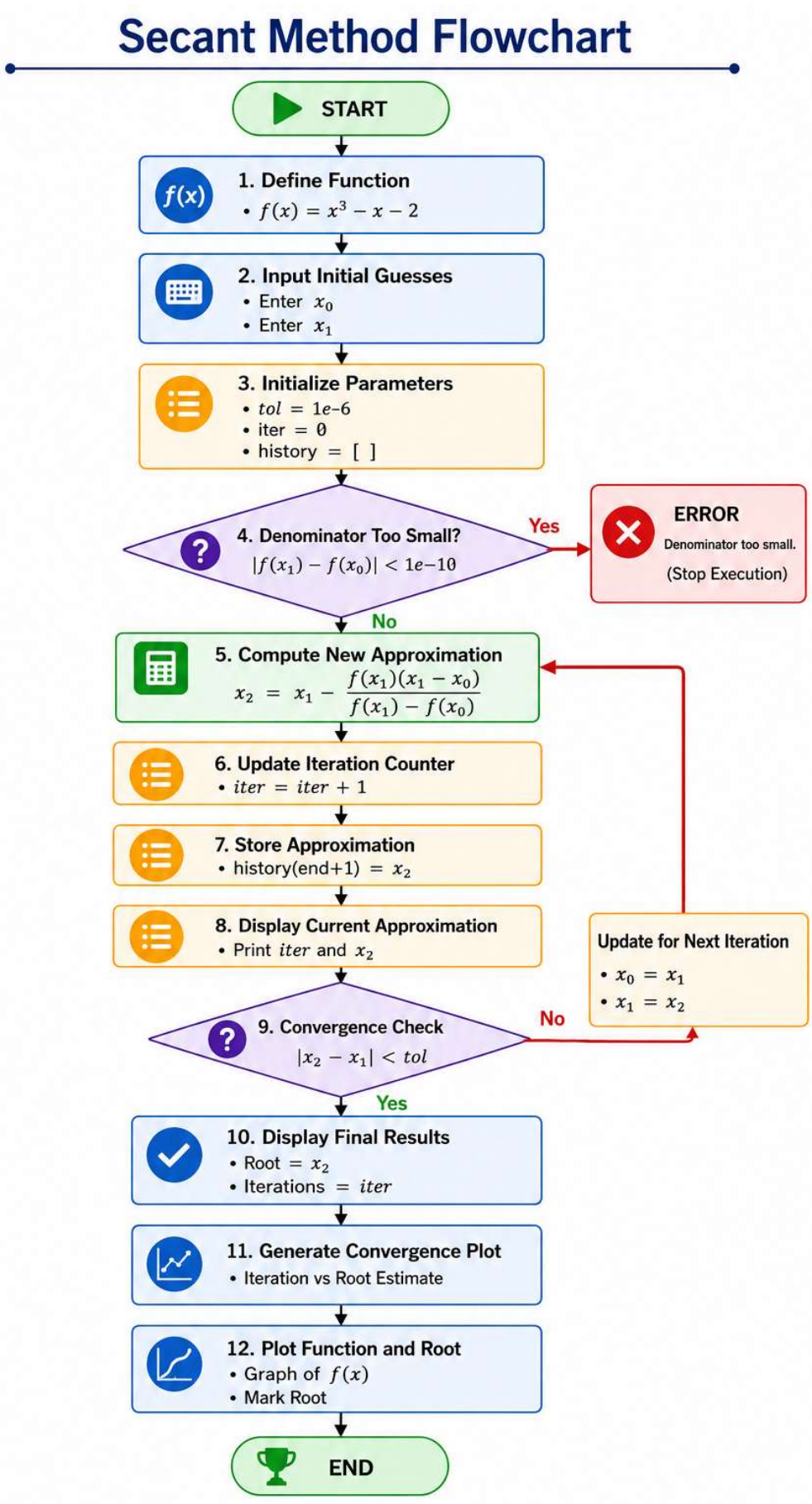
in 7 iterations using the initial guesses

$$x_0 = 1, x_1 = 2$$

Although it required more iterations than the Newton-Raphson Method (7 versus 3), it still converged significantly faster than the Bisection Method (7 versus 20) while avoiding derivative evaluation.

The Secant Method therefore provides a practical compromise between convergence speed and implementation simplicity.

5 FLOWCHART


Secant Method Flowchart
START
1. Define Function
• $f(x) = x^3 - x - 2$
2. Input Initial Guesses
• Enter x_0
• Enter x_1
3. Initialize Parameters
• $tol = 1e{-}6$
• iter = 0
• history = [ ]
4. Denominator Too Small?
$|f(x_1) - f(x_0)| < 1e{-}10$
Yes
ERROR
Denominator too small.
(Stop Execution)
No
5. Compute New Approximation
$x_2 = x_1 - \frac{f(x_1)(x_1 - x_0)}{f(x_1) - f(x_0)}$
6. Update Iteration Counter
• $iter = iter + 1$
7. Store Approximation
• history(end+1) = x_2
8. Display Current Approximation
• Print $iter$ and x_2
Update for Next Iteration
• $x_0 = x_1$
• $x_1 = x_2$
9. Convergence Check
$|x_2 - x_1| < tol$
No
Yes
10. Display Final Results
• Root = x_2
• Iterations = $iter$
11. Generate Convergence Plot
• Iteration vs Root Estimate
12. Plot Function and Root
• Graph of $f(x)$
• Mark Root
END

6 SECANT METHOD CODE

```
% Define the nonlinear function
f = @(x) x.^3 - x - 2;

% Initial Guesses
x0 = input('Enter Initial Guess 1: ');
x1 = input('Enter Initial Guess 2: ');

% Tolerance
tol = 1e-6;

% Counter
iter = 0;

% Store iterations
history = [];

fprintf('Iterations\t x\n')

while true

    if abs(f(x1)-f(x0)) < 1e-10
        error('Denominator too small.')
    end

    x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0));

    iter = iter + 1;

    history(end+1) = x2;

    fprintf('%d\t\t %.8f\n', iter, x2)

    if abs(x2-x1)<tol
        break;
    end

    x0 = x1;
    x1 = x2;

end

fprintf('Iterations = %d\n', iter)
fprintf('Root = %.6f\n', x2)

figure

plot(1:length(history), history, '-o', 'LineWidth', 2, 'MarkerSize', 8)
grid on

xlabel('Iteration Number')
ylabel('Approximate Root Value')
title('Secant Method Iteration Convergence')

for i = 1:length(history)
    text(i, history(i), sprintf('  x_%d', i))
end
```

```
figure

x = 0:0.01:3;
plot(x, f(x), 'b', 'LineWidth', 2)
hold on
grid on

xline(0, 'k', 'LineWidth', 1.5)
yline(0, 'k', 'LineWidth', 1.5)

plot(x2, f(x2), 'ro', 'MarkerSize', 10, 'LineWidth', 2)
text(x2+0.05, f(x2)+0.5, 'Root')

xlabel('x')
ylabel('f(x)')
title('Secant Method Root Finding')
legend('f(x)=x^3-x-2', 'x-axis', 'y-axis', 'Root')
```

7 OUTPUT

7.1 INITIAL VALUES

Parameter	Value
Initial Guess 1 (x_0)	1
Initial Guess 2 (x_1)	2
Tolerance	1×10^{-6}

7.2 CONVERGENCE TABLE

Table 1 Convergence history of the Secant Method for f(x)

Iteration	Approximation
1	1.33333333
2	1.46268657
3	1.53116943
4	1.52092642
5	1.52137632
6	1.52137971
7	1.52137971

7.3 GRAPH

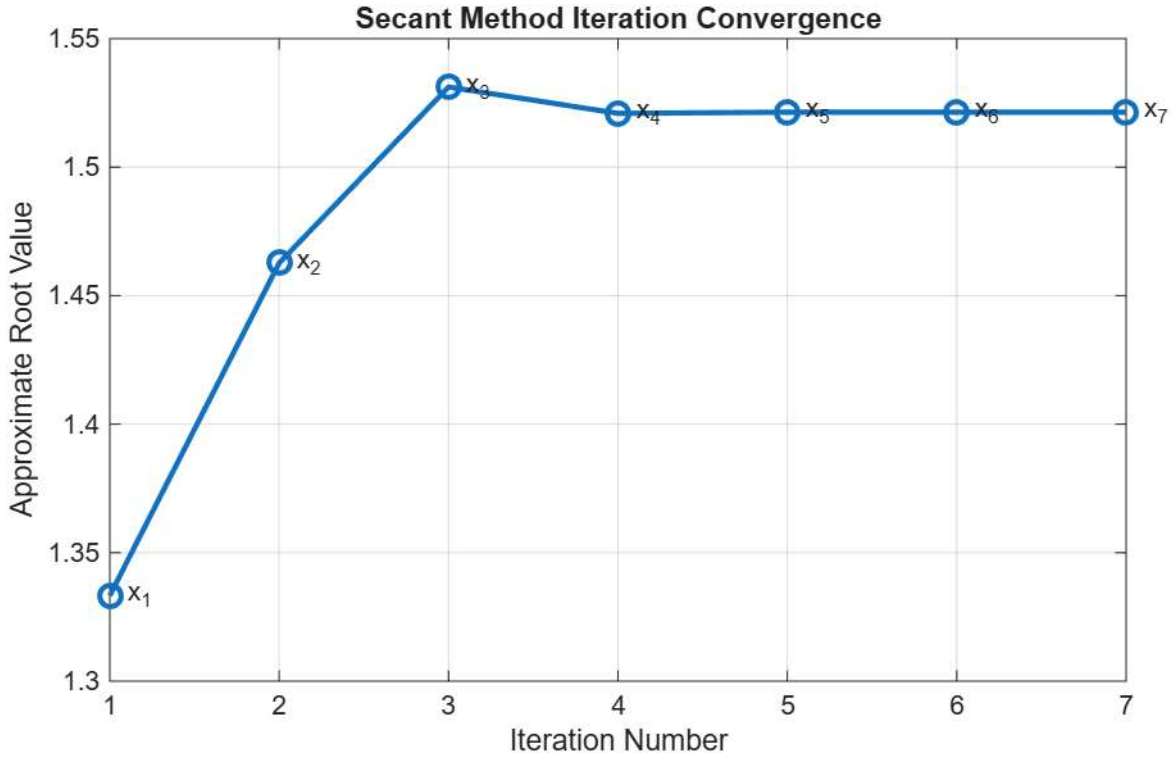


Figure 1 *Iteration convergence plot illustrating the progression of Secant Method approximations toward the root of the nonlinear equation f(x)*

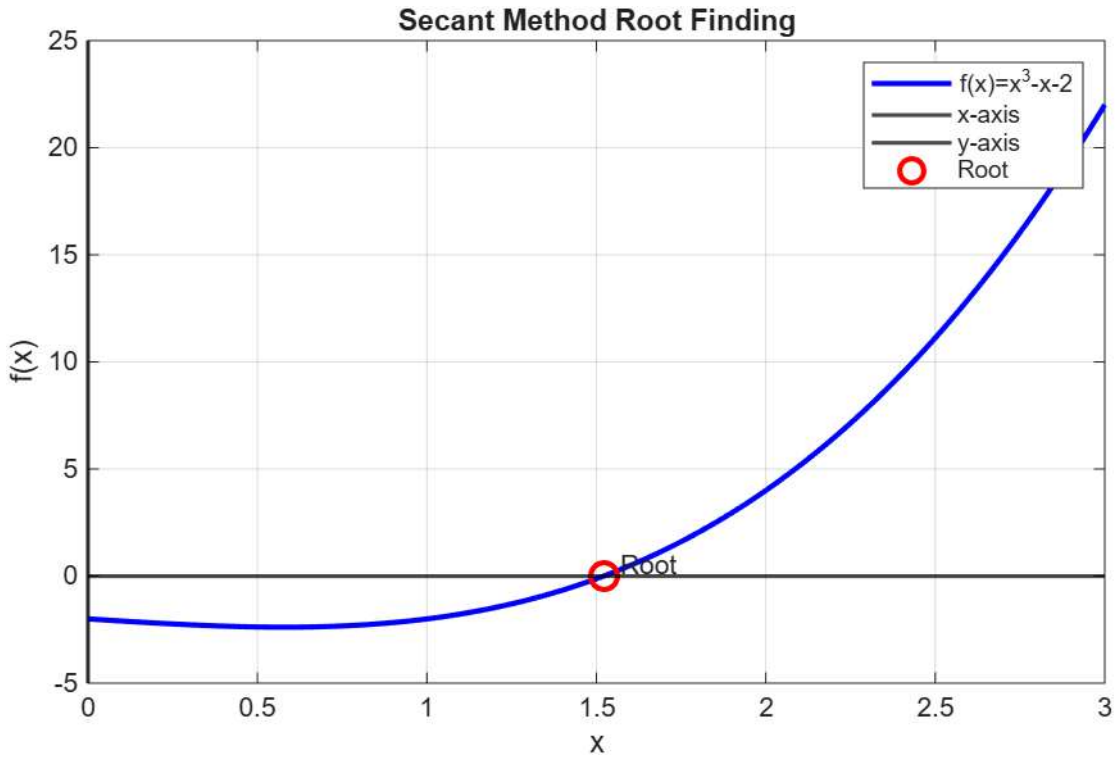


Figure 2 Function plot of f(x) with the computed root highlighted, confirming the numerical solution obtained using the Secant Method.

8 ROBUSTNESS TESTING

To examine the reliability of the Secant Method, the program was tested using different pairs of initial guesses for the same nonlinear equation:

$$f(x) = x^3 - x - 2$$

The purpose of this testing was to observe how the method behaves when the initial guesses are close to the root, moderately far from the root, widely separated, or far from the actual solution.

Case	Initial Guesses	Iterations	Root	Observation
1	(1, 2)	7	1.521380	Fast and stable convergence
2	(0, 2)	10	1.521380	Converged successfully with some early oscillation
3	(-5, 5)	11	1.521380	Larger jumps occurred before convergence
4	(10, 20)	14	1.521380	Far initial guesses caused slower convergence

The results show that the Secant Method successfully converged to the same root for all tested initial guess pairs. The fastest convergence occurred for the pair (1,2), which is close to the actual root and required only 7 iterations.

For the pair (0,2), the method still converged successfully but required 10 iterations. Some early movement away from the final root was observed before the approximations stabilized.

For the pair (-5,5), the method required 11 iterations. This case produced larger jumps in the early iterations because the initial guesses were widely separated.

The slowest convergence occurred for the pair (10,20), which required 14 iterations. Since both initial guesses were far from the actual root, the method needed more iterations before approaching the root accurately.

Overall, the robustness testing shows that the Secant Method is capable of converging from a variety of initial guesses. However, its convergence speed depends strongly on how close the initial guesses are to the actual root. Good initial guesses lead to faster convergence, while poor or distant initial guesses increase the number of iterations.

9 COMPARISON WITH OTHER METHODS

The Bisection, Newton-Raphson, and Secant Methods were implemented in MATLAB to solve

$$f(x) = x^3 - x - 2$$

All three methods converged to the same root:

$$x = 1.521380$$

However, their convergence characteristics differed significantly.

Feature	Bisection	Newton-Raphson	Secant
Root Obtained	1.521380	1.521380	1.521380
Tolerance	1×10^{-6}	1×10^{-6}	1×10^{-6}
Iterations	20	3	7
Uses Derivative	No	Yes	No
Requires Initial Interval	Yes	No	No
Requires Initial Guess	No	Yes	Yes (two guesses)
Convergence Rate	Linear	Quadratic	Superlinear
Guaranteed Convergence	Yes	Not always	Not always
Computational Efficiency	Moderate	High	High

For this problem, the Secant Method reduced the iteration count by approximately 65% compared with the Bisection Method, while avoiding derivative evaluation.

10 RESULTS

The MATLAB implementation successfully determined the root of the nonlinear equation

$$f(x) = x^3 - x - 2$$

using the Secant Method.

Starting from the initial guesses

$$x_0 = 1, x_1 = 2$$

the algorithm converged to

$$x = 1.521380$$

within 7 iterations while satisfying the specified convergence tolerance of

$$1 \times 10^{-6}$$

The convergence table demonstrated the progressive refinement of the root approximation, while the graphical visualization confirmed that the computed root corresponds to the x-intercept of the function.

Compared with the Bisection Method, the Secant Method achieved the same root value while requiring substantially fewer iterations. Although the Newton-Raphson Method converged more rapidly, the Secant Method eliminated the need for derivative evaluation.

Quantity	**Value**
Function	$x^3 - x - 2$
Initial Guesses	(1,2)
Computed Root	1.521380
Tolerance	1×10^{-6}
Iterations	7
Method	Secant Method

The iteration convergence plot further illustrates the rapid stabilization of successive approximations toward the final root value.

11 CONCLUSION

A MATLAB-based Secant Method root solver was successfully developed, implemented, and tested.

The program utilized user-defined initial guesses, convergence monitoring, numerical stability checks, and graphical visualization to accurately determine the root of a nonlinear equation.

For the test function

$$f(x) = x^3 - x - 2$$

the solver converged to the root

$$x = 1.521380$$

in 7 iterations using the initial guesses $x_0 = 1$ and $x_1 = 2$.

Comparison with the Bisection and Newton-Raphson Methods demonstrated that the Secant Method provides an effective balance between computational efficiency and implementation simplicity. Although it converged more slowly than Newton-Raphson, it achieved significantly faster convergence than Bisection while avoiding derivative calculations.

Overall, this project strengthened understanding of derivative-free numerical methods, convergence behavior, iterative algorithms, and MATLAB programming for engineering computation.